



Exploratory Data Analytics

Unit-3

Variable Discretization and Working with Outliers

Department of Computer Science and Engineering(AIML&DS) Academic Year - 2026-27

Contents

- Variable Discretization:- divide the variables into equal intervals,
- perform discretization followed by categorical encoding.
- Working with outliers:- Outliers means?,
- trimming outliers,
- capping the variables at arbitrary max and min values,
- performing zero coding.

3

COURSE OUTCOME (CO)

CO2 - Interpret different EDA techniques to gain insights, identify patterns, detect missing values, outliers, and understand the underlying structure of a real-world datasets.



What is Variable Discretization?

Discretization, also known as binning, is the process of transforming continuous numerical variables into discrete categorical features.

It involves dividing the range of a continuous variable into intervals (bins) and assigning data points to these bins based on their values.

Why Binning Is Used in EDA

1. Simplifies visualization

Plotting 20 individual marks (34, 39, 42, 45...) on a chart is cluttered and hard to read. Plotting 5 bins ("21–40", "41–60"...) as a simple bar chart instantly shows: most students scored in the 41–80 range.

2. Reduces noise

Whether a student got 61 or 65 doesn't matter much — they're basically "similar performers." Binning lumps them together so tiny score differences don't distract from the bigger trend.

3. Handles outliers

Suppose one student scored just 5 marks (way below everyone else). Instead of that one number stretching your whole graph's scale, it just sits quietly in the "0–20" bin.



Why Binning Is Used in EDA

4. Reveals non-linear patterns

Maybe you notice something interesting: students scoring 81–100 are almost all from one particular class section. You might miss this pattern staring at 20 raw numbers, but it jumps out once grouped and compared.

5. Easier to interpret/communicate

Telling your principal "7 students scored in the 61–80 range" is much clearer than reading out 7 individual scores like 62, 65, 68, 70, 74, 78, 79.

6. Helps with skewed distributions

If most students scored 40–60 but a couple scored 95–99 (skewed data), binning groups the majority together so the comparison ("most are average, a few are exceptional") is easy to see — instead of the top scorers distorting an average.

7. Better for some ML models

If you're building a model to predict "Pass/Fail risk," a decision-tree-based model might work better learning from "score bin: 41–60" than from the raw number "58" — grouped categories can be easier for it to split decisions on.



Let's use **one example throughout**: exam marks of 20 students (out of 100).

Raw data (no binning):

34, 39, 42, 45, 48, 51, 55, 58, 60, 62, 65, 68, 70, 74, 78, 82, 85, 90, 95, 99

After binning (grouping into ranges of 20):

Bin (Range)	Students in this bin
0–20	0
21–40	2
41–60	6
61–80	7
81–100	5

Which Data Needs Binning?

1. Continuous variables with wide ranges

Example: House prices ranging from ₹5 lakh to ₹5 crore. That's a huge spread — hard to plot or compare directly.
→ Bin them into: "Under 20L", "20L–50L", "50L–1Cr", "1Cr+". Now you can easily say "most houses fall in the 20L–50L range" instead of drowning in exact prices.

2. Skewed distributions

Example: Employee salaries — most people earn ₹30k–60k, but the CEO earns ₹50 lakh. This one huge number "skews" (pulls) the average up, making it misleading.
→ Bin salaries into ranges: "10–50k", "50k–1L", "1L–5L", "5L+". Now the CEO's salary just sits in its own bin instead of distorting the whole picture — you can see clearly that most employees are actually in the lower ranges.

3. Variables with outliers

Example: Delivery times for 100 orders — most take 20–40 minutes, but one order got stuck and took 6 hours (360 minutes) due to a mistake.

- If you plot raw minutes, that one outlier stretches your entire graph, making the "normal" 20–40 min pattern look tiny and squished.
- Bin into: "0–30 min", "30–60 min", "60+ min". The 6-hour order just falls into "60+ min" without ruining your chart's scale.

4. High-cardinality numerical data

(Cardinality = number of different unique values)

Example: Customer ages ranging from 1 to 99 — that's potentially 99 different exact values in your data.

- Trying to analyze "how many customers are exactly 34 vs exactly 35" isn't very useful — there's too much granularity.
- Bin into: "0–18", "19–35", "36–55", "56+". Now instead of 99 possible values, you only have 4 simple groups — much easier to analyze and spot trends like "36–55 age group buys the most."

Data That Usually Doesn't Need Binning:

1. Already categorical data

Example: A column called "Gender" with values like "Male", "Female", "Other" — or a column called "City" with values like "Mumbai", "Delhi", "Pune".

→ These are already in distinct groups. There's nothing to "bin" here — binning is for converting continuous numbers into groups, and this data is already grouped by nature.

2. Discrete numerical data with few unique values

Example: "Number of children" in a family — most values are just 0, 1, 2, 3, or 4.

→ Since there are already only a handful of possible values, grouping them further (like "0–2" and "3+") doesn't really simplify anything — you'd just be squashing already-simple data into fewer, less useful categories.

3. Numeric IDs or codes

Example: Customer ID (like 10234, 10235, 10236...) or a Pin Code (like 411001).

→ These numbers don't represent a *quantity* — ID 10235 isn't "greater than" or "more" than ID 10234 in any meaningful sense. They're just labels. Binning them (like "10000–10500") would be meaningless since there's no real numeric relationship to group by.

4. Time series data

Example: Daily stock prices over the last 2 years, or hourly temperature readings.

→ You *can* bin time series data (e.g., grouping into "Morning/Afternoon/Evening" or "Weekday/Weekend"), but it usually needs special handling — because the **order and trend over time matters** (yesterday's price affects today's). Careless binning can destroy that time-based pattern, so it's used more cautiously here than with typical numeric data.

In short: Don't bin data that's already grouped, has too few values to begin with, is just a label/ID (not a real quantity), or is time-ordered data where sequence matters more than range.

Discretization

Discretization is the process of converting a **continuous variable** into a **discrete (categorical) one** by dividing its range of values into a finite number of intervals (bins).

It's essentially the formal/technical term for **binning** — you'll often see the two used interchangeably in data science.

Example

Continuous variable: **Temperature** (in °C) — e.g., 12.5, 18.3, 25.7, 33.1...

Discretized version:

Now "25.7°C" simply becomes the category "**Warm.**"

Range	Label
< 15	Cold
15–25	Mild
25–35	Warm
> 35	Hot

Discretization vs Binning

They're **very closely related** — so closely that people often use them interchangeably. But there's a subtle difference in scope:

Discretization (the broader concept)

The **overall process** of converting continuous data into discrete/categorical form.

It's the **umbrella term** — the *goal*.

Binning (one specific technique)

A **method** used to achieve discretization — specifically, grouping values into **ranges/intervals (bins)**.

It's one of the ways to do discretization.

Simple Analogy

Think of it like:

- **Discretization** = "Cooking a meal" (the overall goal/process)
- **Binning** = "Frying" (one specific technique to cook)



Types of Discretization

Types of Discretization

1. Unsupervised Discretization
 - a. Equal-width binning
 - b. Equal-frequency binning
2. Supervised Discretization
 - a. Decision tree-based binning
 - b. ChiMerge / Chi2-based
 - c. Entropy-based binning

I. Equal-Width Binning

- **Definition:** Divides the range of values into N intervals of **equal size (width)**.
- **Use case:** When variable distribution is uniform or when simplicity is preferred.
- **How it works:**
 - Formula:

$$\text{Bin Width} = \frac{\text{Max value} - \text{Min value}}{\text{Number of bins}}$$

Example

Suppose you have exam marks ranging from **0 to 100**, and you want **5 bins**.

Bin Width= $\frac{100-0}{5}=20$

So the bins become:

Each bin covers the **same 20-mark width** — regardless of how many students actually fall into each one.

Now say you have these 10 students' marks:

5, 15, 22, 25, 30, 45, 55, 62, 88, 95

Notice: the bins are **equal in width (20 marks each)**, but the **number of students in each bin is uneven**.

Bin	Range
Bin 1	0–20
Bin 2	20–40
Bin 3	40–60
Bin 4	60–80
Bin 5	80–100

Bin	Students Falling In It
0–20	5, 15 → 2 students
20–40	22, 25, 30 → 3 students
40–60	45, 55 → 2 students
60–80	62 → 1 student
80–100	88, 95 → 2 students

Pros

- Simple and easy to compute
- Good for data that's fairly evenly spread out
- Preserves the original scale/meaning of values (e.g., "0–20 marks" is intuitive)

Cons

- **Sensitive to outliers** — one extreme value can stretch the range and create empty or nearly-empty bins.
Example: If one student scored 5 and another scored 500 (data entry error), the bin width becomes huge, and most real students get squeezed into just 1–2 bins.
- Can result in **uneven bin populations** — some bins packed with data, others nearly empty — especially with skewed data.

When to use it

Best when your data is **roughly uniform/evenly spread** without major outliers or skew. If your data is skewed or has extreme values, **Equal-Frequency Binning** (where each bin has the same *number* of data points instead of the same *width*) is usually a better choice.



```
import pandas as pd
import numpy as np

# Sample data: exam marks
marks = [5, 15, 22, 25, 30, 45, 55, 62, 88, 95]

# Equal-width binning into 5 bins
bins = pd.cut(marks, bins=5)

df = pd.DataFrame({'Marks': marks, 'Bin': bins})
print(df)

# With custom labels
labels = ['0-20', '20-40', '40-60', '60-80', '80-100']
df['Bin_Label'] = pd.cut(marks, bins=5, labels=labels)
print(df)
```

	Marks	Bin
0	5	(4.91, 23.0]
1	15	(4.91, 23.0]
2	22	(4.91, 23.0]
3	25	(23.0, 41.0]
4	30	(23.0, 41.0]
5	45	(41.0, 59.0]
6	55	(41.0, 59.0]
7	62	(59.0, 77.0]
8	88	(77.0, 95.0]
9	95	(77.0, 95.0]

Key points about the code:

- `pd.cut()` is the standard pandas function for **equal-width** binning (splits the range evenly, regardless of how many values fall in each bin).
- `bins=5` tells it to create 5 equal-width intervals automatically based on min/max of the data.
- You can pass `labels=[...]` to name the bins instead of showing raw ranges.
- For **manual bin edges** (custom width per bin), pass a list instead of a number:

python

```
pd.cut(marks, bins=[0, 20, 40, 60, 80, 100])
```

- For **equal-frequency binning** instead (same count per bin, not same width), use `pd.qcut()` instead of `pd.cut()`.

That "Bin" column just shows **which range/group each mark falls into**, written in interval notation.

Example: (4.91, 23.0]

This means: **"greater than 4.91, up to and including 23.0"**

- (= does NOT include this number (exclusive)
-] = DOES include this number (inclusive)

So a mark like **15** falls into (4.91, 23.0] because 15 is between 4.91 and 23.0.

Why 4.91 and not 0, 20, 40...?

Because `pd.cut(marks, bins=5)` automatically calculates bin edges from your **actual min and max** in the data (5 to 95), not a clean 0–100 scale. It also adds a tiny buffer below the minimum so the lowest value doesn't get excluded.

Bin Width= $\frac{95-5}{5}=18$

So the 5 automatic bins become roughly:

Each is **18 marks wide**, just starting from 5 (your actual minimum) instead of 0.

This is why the code also showed a `Bin_Label` column — to replace those confusing interval numbers with simple readable names like "0–20", "20–40", etc., so instead of seeing (4.91, 23.0] you just see "0–20".

If you want **clean round bins** (0–20, 20–40...) instead of these auto-calculated odd numbers, just tell `pd.cut` the exact edges yourself:

```
python
pd.cut(marks, bins=[0, 20, 40, 60, 80, 100])
```

That will give clean bins like (0, 20], (20, 40], etc.

Bin	Range
Bin 1	(4.91, 23.0]
Bin 2	(23.0, 41.0]
Bin 3	(41.0, 59.0]
Bin 4	(59.0, 77.0]
Bin 5	(77.0, 95.0]

2. Equal-Frequency (Quantile) Binning



Definition: Divides data such that each bin has **approximately the same number of observations**.



Use case: Better for skewed distributions.



How it works:



Uses `qcut()` in pandas, which bins based on quantiles.

Example- Equal-Frequency (Quantile) Binning

```
import pandas as pd

marks = [5, 15, 22, 25, 30, 45, 55, 62, 88, 95]

# Equal-Frequency (Quantile) Binning - 5 bins, ~2 values each
bins = pd.qcut(marks, q=5)

df = pd.DataFrame({'Marks': marks, 'Bin': bins})
print(df)

# With readable custom labels instead of raw ranges
labels = ['Very Low', 'Low', 'Medium', 'High', 'Very High']
df['Bin_Label'] = pd.qcut(marks, q=5, labels=labels)
print(df)
```

	Marks	Bin	Bin_Label
0	5	(4.999, 20.6]	Very Low
1	15	(4.999, 20.6]	Very Low
2	22	(20.6, 28.0]	Low
3	25	(20.6, 28.0]	Low
4	30	(28.0, 49.0]	Medium
5	45	(28.0, 49.0]	Medium
6	55	(49.0, 67.2]	High
7	62	(49.0, 67.2]	High
8	88	(67.2, 95.0]	Very High
9	95	(67.2, 95.0]	Very High

Notice:

- Each bin has exactly **2 students** (equal frequency) ✓
- The bin **widths differ**: 15.6, 7.4, 21, 18.2, 27.8 — not equal, because **qcut** prioritizes equal *count*, not *range*.
- **q=5** means 5 quantile-based bins; you can also pass explicit quantile cut points like **q=[0, 0.2, 0.4, 0.6, 0.8, 1]** for finer control.

Example- Equal-Frequency (Quantile) Binning

```
import pandas as pd

df = pd.DataFrame({'Score': [12, 45, 67, 23, 89, 55, 38, 90]})

# Equal-Width Binning (4 bins of equal range)
df['EqualWidthBin'] = pd.cut(df['Score'], bins=4, labels=['Low', 'Mid-Low', 'Mid-High',

# Equal-Frequency Binning (4 bins with ~equal count per bin)
df['EqualFreqBin'] = pd.qcut(df['Score'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

print(df)
```

	Score	EqualWidthBin	EqualFreqBin
0	12	Low	Q1
1	45	Mid-Low	Q2
2	67	Mid-High	Q3
3	23	Low	Q1
4	89	High	Q4
5	55	Mid-High	Q3
6	38	Mid-Low	Q2
7	90	High	Q4

3. Custom Binning

Custom Bin	Range
Child	0–12
Teen	13–19
Adult	20–59
Senior	60+

Unlike Equal-Width or Equal-Frequency binning (which are calculated mathematically), **Custom Binning** means **you manually define the bin edges/ranges yourself** — based on domain knowledge, business rules, or meaningful real-world thresholds, not a formula.

Example

Instead of letting Python auto-calculate bin widths or quantiles, you decide the age groups that make sense in real life:

These boundaries aren't based on equal width or equal count — they're based on **what actually makes sense** for age categories.



```
import pandas as pd

ages = [5, 12, 15, 19, 25, 34, 45, 60, 65, 90]
df = pd.DataFrame({'Age': ages})

# Custom bin edges
bin_edges = [0, 12, 19, 59, 120]
labels = ['Child', 'Teen', 'Adult', 'Senior']

df['AgeGroup'] = pd.cut(df['Age'], bins=bin_edges, labels=labels)
print(df)
```

	Age	AgeGroup
0	5	Child
1	12	Child
2	15	Teen
3	19	Teen
4	25	Adult
5	34	Adult
6	45	Adult
7	60	Senior
8	65	Senior
9	90	Senior

Key Points

- Uses `pd.cut()` — same function as equal-width binning — but you pass **your own custom bins list** instead of just a number.
- Bin widths can be **completely different sizes** (Child = 0–12 is 12 years wide, Adult = 20–59 is 39 years wide) — because they're based on meaning, not math.
- **Best when:** you have domain expertise and want bins that reflect real-world categories (age brackets, income tax slabs, credit score ranges, BMI categories, etc.) rather than statistically "even" splits.

Custom Binning Example

- **Definition:** You define the bin edges manually, based on **domain knowledge** or other criteria.
- **Use case:** When certain ranges are known to have meaning (e.g., age groups).

```
# Custom bin edges and labels
bins = [0, 30, 60, 100]
labels = ['Low', 'Medium', 'High']
df['CustomBin'] = pd.cut(df['Score'], bins=bins, labels=labels)

print(df)
```

	Score	EqualWidthBin	EqualFreqBin	CustomBin
0	12	Low	Q1	Low
1	45	Mid-Low	Q2	Medium
2	67	Mid-High	Q3	High
3	23	Low	Q1	Low
4	89	High	Q4	High
5	55	Mid-High	Q3	Medium
6	38	Mid-Low	Q2	Medium
7	90	High	Q4	High

- Working with outliers:- Outliers means?,
- trimming outliers,
- capping the variables at arbitrary max and min values,
- performing zero coding.

What are Outliers?

What are Outliers?

- An **outlier** is a data point that is **significantly distant** or **inconsistent** compared to other values in the dataset.

Definition:

- **Mathematically:** A value that lies far outside the typical range.
- **Statistically:** Can be due to:
 - Natural variation, Data entry error, Rare event

Example:

- Observations = [20, 22, 23, 25, 26, 100]
- Here, **100** is an outlier — it's way higher than the others.

Why Do Outliers Matter?

Impact	Description
Skew Summary Stats	Mean and standard deviation become unreliable.
Affect ML Models	Especially sensitive ones like regression, SVM, and k-means.
Increase Error Rates	Can lead to overfitting or underfitting.
Mislead Visualizations	May stretch or flatten plots like boxplots or histograms.

Techniques to Detect Outliers

- **Z-Score Method**
- **Box Plot / IQR Method (Interquartile Range)**

Common Methods to Highlight Outliers

1. Using Z-score (Standard Score)

- Formula:

$$Z = \frac{(X - \mu)}{\sigma}$$

where:

- X = data point
- μ = mean
- σ = standard deviation
- If:
 - $|Z| > 3 \rightarrow$ Typically considered an **outlier**

```
import pandas as pd
from scipy import stats

# Sample data
data = pd.DataFrame({'score': [20, 22, 23, 25, 30, 100]})

# Calculate Z-score
data['zscore'] = stats.zscore(data['score'])

# Identify outliers using Z-score threshold > 2
data['is_outlier'] = data['zscore'].abs() > 2

# Display entire DataFrame with outlier flag
print(data)
```

	score	zscore	is_outlier
0	20	-0.584925	False
1	22	-0.514734	False
2	23	-0.479639	False
3	25	-0.409448	False
4	30	-0.233970	False
5	100	2.222716	True

Z-Score Method

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- Z = Z-score
- X = value from the dataset
- μ = mean of the dataset
- σ = standard deviation of the dataset

- Measures how many **standard deviations** a point is from the mean.
- If **$Z = 0$** → the value is **exactly at the mean**.
- If **$Z = 1$** → the value is **1 standard deviation above the mean**.
- If **$Z = -2$** → the value is **2 standard deviations below the mean**.

Example

For the dataset: [10, 12, 15, 17, 22, 90]

1. Mean (μ) = 27.67
2. Standard deviation (σ) $\approx$ 29.33
3. For value $X = 90$:

$$Z = \frac{90 - 27.67}{29.33} \approx 2.13$$

```

import pandas as pd
from scipy.stats import zscore

# Data
data = pd.DataFrame({'Value': [10, 12, 15, 17, 22, 90]})

# Calculate z-scores
data['zscore'] = zscore(data['Value'])

# Print z-scores for reference
print(data)

# Detect outliers using Z-score threshold
outliers = data[data['zscore'].abs() > 2.0]

# Output
print("\nOutliers based on Z-score (threshold = 2.0):")
print(outliers)

```

	Value	zscore
0	10	-0.627934
1	12	-0.556847
2	15	-0.450217
3	17	-0.379130
4	22	-0.201413
5	90	2.215540

```

Outliers based on Z-score (threshold = 2.0):
   Value  zscore
5     90  2.21554

```

IQR Method (Interquartile Range)

2. Using IQR (Interquartile Range)

- $IQR = Q3 - Q1$
- Outlier if:
 - $Value < Q1 - 1.5 \times IQR$
 - $Value > Q3 + 1.5 \times IQR$

```
import pandas as pd

# Create DataFrame
data = pd.DataFrame({'Value': [10, 12, 15, 17, 22, 90]})

# Calculate Q1 and Q3
Q1 = data['Value'].quantile(0.25)
Q3 = data['Value'].quantile(0.75)

# Calculate IQR
IQR = Q3 - Q1
|
# Define lower and upper bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = data[(data['Value'] < lower_bound) | (data['Value'] > upper_bound)]

# Print results
print("Outliers:")
print(outliers)
```

```
Outliers:
  Value
5     90
```

3. Box Plot Visualization

- Boxplot shows:
 - Median, Q1, Q3
 - "Whiskers" (normal range)
 - Dots beyond whiskers = outliers

Box Plot Components:

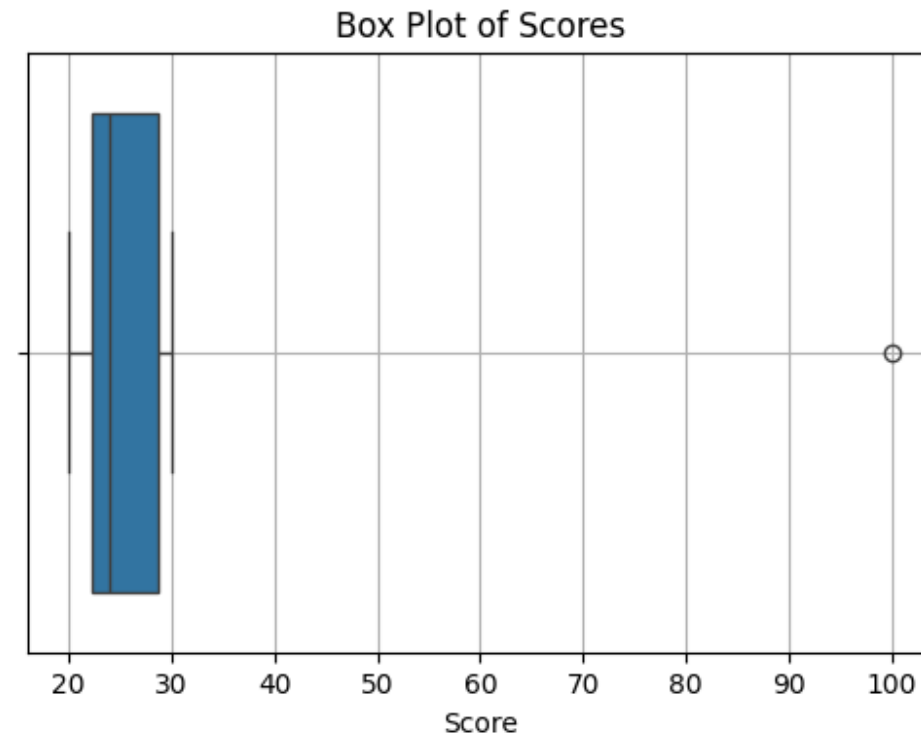
- **Box:** Middle 50% of data (Interquartile Range or IQR).
- **Line in Box:** Median value.
- **Whiskers:** Extend to min and max values **within** $1.5 \times \text{IQR}$.
- **Circle (dot): Outlier** (a value that is far from the others).

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = pd.DataFrame({'score': [20, 22, 23, 25, 30, 100]})

# Create a box plot
plt.figure(figsize=(6, 4))
sns.boxplot(x=data['score'])

# Add labels and title
plt.title("Box Plot of Scores")
plt.xlabel("Score")
plt.grid(True)
plt.show()
```



Why 100 is an Outlier?

Let's compute IQR:

- Data: [20, 22, 23, 25, 30, 100]

- Q1 (25th percentile) = 22

- Q3 (75th percentile) = 30

- $\text{IQR} = \text{Q3} - \text{Q1} = 30 - 22 = 8$

- Upper limit = $\text{Q3} + 1.5 \times \text{IQR} = 30 + 12 = 42$

Value $100 > 42 \rightarrow$ **Outlier** (plotted as a circle).

Outlier Handling Techniques

- 1. Trimming (Removing)**
- 2. Capping (Winsorization)**
- 3. Zero Coding**

Trimming (Removing)

- **What is Trimming (Removing)?**
- **Trimming**, also called **outlier removal**, is the technique of **removing rows** that contain outliers based on some threshold (e.g., using IQR or z-score methods).
- **How It Works:**
- **Calculate bounds** using IQR (Interquartile Range):
 - Lower Bound = $Q1 - 1.5 \times IQR$
 - Upper Bound = $Q3 + 1.5 \times IQR$
- **Keep only** the values within bounds.

```

import pandas as pd

# Sample data with outliers
data = pd.DataFrame({
    'Value': [10, 12, 11, 13, 14, 12, 200, 15, 13, 12, 11, 210]
})

# Step 1: Calculate IQR
Q1 = data['Value'].quantile(0.25)
Q3 = data['Value'].quantile(0.75)
IQR = Q3 - Q1

# Step 2: Calculate bounds
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Step 3: Remove (trim) outliers
data_trimmed = data[(data['Value'] >= lower) & (data['Value'] <= upper)]

print("\nTrimmed Data (No Outliers):\n", data_trimmed)

```

Trimmed Data (No Outliers):

	Value
0	10
1	12
2	11
3	13
4	14
5	12
7	15
8	13
9	12
10	11

Pros & Cons:

Pros

Simple & easy to implement
Useful for extreme outliers

Cons

May remove useful information
Loss of data, especially if dataset is small

When to Use?

- When dataset is **large** enough that removing some rows won't hurt.
- When **extreme outliers** are clearly errors or irrelevant.



Capping (Winsorization)

- **What is Capping (Winsorization)?**
- **Capping**, also known as **Winsorization**, is an outlier-handling technique where **extreme values** (outliers) are **replaced** with the nearest acceptable minimum or maximum threshold.
- Unlike trimming, **no data is removed** — instead, outliers are **converted** into boundary values.
- **How It Works:**
- **Determine bounds** using IQR or z-score:
 - Lower Bound = $Q1 - 1.5 \times IQR$
 - Upper Bound = $Q3 + 1.5 \times IQR$
- **Clip values** outside the bounds:
 - Values $<$ Lower Bound $\rightarrow$ set to Lower Bound
 - Values $>$ Upper Bound $\rightarrow$ set to Upper Bound

```
import pandas as pd

# Sample data
data = pd.DataFrame({
    'Value': [10, 12, 11, 13, 14, 12, 200, 15, 13, 12, 11, 210]
})

# Step 1: Calculate IQR
Q1 = data['Value'].quantile(0.25)
Q3 = data['Value'].quantile(0.75)
IQR = Q3 - Q1

# Step 2: Set bounds
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Step 3: Apply capping (Winsorization)
data['Capped'] = data['Value'].clip(lower=lower, upper=upper)

print("Original Data:\n", data['Value'].tolist())
print("Capped Data:\n", data['Capped'].tolist())
```

Original Data:

[10, 12, 11, 13, 14, 12, 200, 15, 13, 12, 11, 210]

Capped Data:

[10, 12, 11, 13, 14, 12, 18, 15, 13, 12, 11, 18]

Pros & Cons:

Pros

Keeps dataset size unchanged
Safer than trimming for small data

Cons

May distort actual extreme values
Needs careful threshold selection

When to Use?

- When you want to **preserve dataset size**.
- When outliers may be **real but extreme**, and you want to **minimize distortion**.



3. Zero Coding

- What is Zero Coding? Zero Coding is a simple technique where outlier values are replaced with 0 or a neutral/default value instead of being removed or capped.
- Purpose:
- To retain all rows in the dataset.
- To neutralize the effect of extreme values.
- Often used when 0 has a valid interpretation, such as absence or no response.

```

import pandas as pd

# Sample data
data = pd.DataFrame({
    'Value': [10, 12, 11, 13, 14, 12, 200, 15, 13, 12, 11, 210]
})

# Step 1: Calculate IQR
Q1 = data['Value'].quantile(0.25)
Q3 = data['Value'].quantile(0.75)
IQR = Q3 - Q1

# Step 2: Determine bounds
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Step 3: Zero Coding
data['ZeroCoded'] = data['Value']
data.loc[(data['Value'] < lower) | (data['Value'] > upper), 'ZeroCoded'] = 0

print(data)

```

	Value	ZeroCoded
0	10	10
1	12	12
2	11	11
3	13	13
4	14	14
5	12	12
6	200	0
7	15	15
8	13	13
9	12	12
10	11	11
11	210	0

Pros & Cons:

Pros

Simple to implement
Keeps dataset size unchanged
Neutralizes influence of outliers

Cons

Can introduce bias (if 0 is meaningful)
May distort statistics (mean, std)
0 might not be contextually appropriate

When to Use?

- When 0 is an acceptable neutral value in the dataset.
- When you want a **quick, minimal-effort solution** to neutralize outliers.
- When you don't want to delete or clip values but just **reduce their impact**.



```

import pandas as pd

data = pd.Series([10,12,13,15,14,1000,11,13,15,9])

Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

trimmedData = data[(data >= lower_bound) & (data <= upper_bound)]

CappedData = data.clip(lower=lower_bound, upper=upper_bound)

zercoded = data.mask((data < lower_bound) | (data > upper_bound), 0)

print("Data after trimming:")
print(trimmedData)

print("Capped Data:")
print(CappedData)

print("Zero-coded Data:")
print(zercoded)

```

Data after trimming:

0	10
1	12
2	13
3	15
4	14
6	11
7	13
8	15
9	9

dtype: int64

Capped Data:

0	10
1	12
2	13
3	15
4	14
5	20
6	11
7	13
8	15
9	9

dtype: int64

Zero-coded Data:

0	10
1	12
2	13
3	15
4	14
5	0
6	11
7	13
8	15
9	9

dtype: int64

Question Bank

1. Explain the process of dividing a continuous variable into equal intervals for discretization. How does this help in data analysis?
2. Describe how discretization followed by categorical encoding is performed on a continuous variable. Provide an example.
3. What are outliers in a dataset? Explain their possible effects on data analysis and modeling.
4. Explain the method of trimming outliers in a dataset. How does this approach differ from capping?
5. What does it mean to cap variables at arbitrary maximum and minimum values? When and why would you use this technique?
6. Define zero coding for outliers and discuss its advantages and limitations.
7. Given a continuous variable, demonstrate how you would discretize it into 4 equal intervals and then perform one-hot encoding on the resulting categorical variable.
8. Compare and contrast trimming, capping, and zero coding as methods to handle outliers. Discuss scenarios where each method might be preferred.